Projet
Outils pour le calcul 2025-2026
—L2 informatique—

Le contrôle de travaux pratiques aura lieu **le jeudi 4 juin de 10 à 12 heures**. Il reposera sur le contenu de ce projet. Il vous est donc conseillé de le travailler dès que possible pour pouvoir répondre aux questions.

Le but de ce projet consiste à compléter la bibliothèque de fonction sur les polynômes réalisées en travaux pratiques. Vous devrez reprendre les programmes et insérer votre code dans les fichiers `Q_poly.ml` et `Q_poly.mli`.

1 Interpolation de Newton

Ce travail est lié aux exercices 5 à 9 de la fiche TD 4. Il est conseillé de bien les travailler et de bien les comprendre avant de commencer à programmer.

1. Écrire la fonction `delta : t -> t` qui réalise l'opérateur Δ, puis la fonction `delta_n : t -> int -> t` qui réalise Δ^n (c'est à dire l'opérateur Δ appliquée n fois).
2. Écrire la fonction `suivant_suite : Q.t list -> Q.t` qui calcule le nombre qui complète une suite d'entiers représentée par une liste.
3. Écrire la fonction `pol_newton : int -> t list` qui retourne la liste des n premiers polynômes de Newton.
4. Écrire la fonction `interpol_newton : Q.t list -> t` qui calcule le polynôme d'interpolation de Newton.

2 Développement de Taylor des fractions rationnelles

On considère une fraction rationelle $F = \frac{P}{1+Q}$ où P et Q sont deux polynômes et $Q(0) = 0$. Un résultat de Taylor stipule que pour un nombre z dont la valeur absolue $|z|$ est suffisament petite, la fraction $F(z)$ peut être approximée par l'évaluation d'un polynôme en z. Plus précisement, si on pose $F_n = \sum_{i=0}^{n} \frac{1}{i!} F^{(i)}(0) X^i$ où $F^{(i)}$ est la dérivée $i$ème de la fonction F, alors il existe $\rho_F > 0$ tel que pour tout z satisfaisant $|z| < \rho_F$ on a

$$\lim_{n \to \infty} F_n(z) = F(z).$$

On veut calculer F_n de trois façons différentes.

2.1 Première méthode

La première méthode consiste à réaliser un calcul direct.

1. Si cela n'a pas déjà été fait, écrire la fonction `derivee : t -> t` permettant de calculer la dérivée d'un polynôme.
2. Proposez un type `frac_rat` permettant d'encoder une fraction rationelle.
3. Écrire la fonction `derivee_frac_rat : frac_rat -> frac_rat` permettant de calculer la dérivée d'une fraction. On rappelle la formule
$$\left(\frac{P}{Q}\right)' = \frac{P'Q - PQ'}{Q^2}.$$
4. Écrire la fonction `test_frac_rat : frac_rat -> bool` vérifiant qu'une fraction est sous la forme $\frac{P}{1+Q}$ avec $Q(0) = 0$.
5. Écrire la fonction `eval_0 : frac_rat -> Q.t` permettant d'évaluer une fraction sous la bonne forme en $z = 0$.
6. En déduire la fonction `dev1 : int -> frac_rat -> t` permettant de calculer F_n pour F passée en paramètre lorsque celle-ci est de la bonne forme.

2.2 Seconde méthode

L'idée est d'utiliser la méthode de Newton vue en cours pour la division de polynômes afin de calculer des approximation de $1/(1+Q(X))$.

1. Écrire la fonction `dev_aux1 : int -> frac_rat -> t` permettant de calculer F_n lorsque $F = \frac{1}{1+Q}$ avec $Q(0) = 0$. Cette fonction devra utiliser la méthode de Newton.
2. En déduire la fonction `dev2 : int -> frac_rat -> t` permettant de calculer F_n lorsque $F = \frac{P}{1+Q}$ avec $Q(0) = 0$.

2.3 Troisième méthode

Si $P = \sum_{i=0}^{n} a_i X^i$ et $Q = \sum_{i=0}^{n} b_i X^i$ sont deux polyômes, on note

$$P \circ Q = P(Q) = \sum_{i=0}^{n} a_i Q(X)^i.$$

Cette opération s'appelle la composition.

1. Écrire la fonction `compose : t -> t -> t` permettant de réaliser la composition de deux polynômes.
2. Notons $H = \frac{1}{1-X}$. Le développement de Taylor à l'ordre n de H est connu et vaut

   $$H(X) = 1 + X + X^2 + \cdots + X^{n-1}.$$

   Si $F = \frac{1}{1+Q}$ avec $Q = 0$ alors le polynôme F_n est obtenu en tronquant le polynôme $F(-Q)$ au degré n. Écrire la fonction `dev_aux2 : int -> frac_rat -> t` utilisant la composition et permettant de calculer F_n dans ce cas.
3. En déduire la fonction `dev3 : int -> frac_rat -> t` permettant de calculer F_n lorsque $F = \frac{P}{1+Q}$ avec $Q(0) = 0$.